

COMPUTER ORGANIZATION

It describes the **design and function of** various units of digital computer that stores and processes information. It also deals with the units of the computer that receives the information from external sources and sends computed results to external destinations.

Computer: It is a fast electronic calculating machine that accepts digitized input information, processes it according to the list of internally stored instructions & produces the resulting output information. The list of internally stored instructions is called as a Computer program & internal storage is called as Computer memory.

Functional units:

A computer consists of five functionally independent parts: *Input, output, ALU, memory & control units* as shown in figure 1.1.

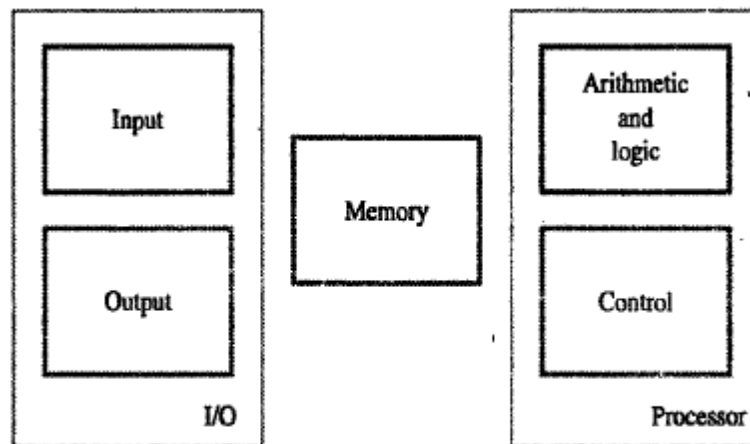


Figure 1.1 Basic functional units of a computer.

The input unit accepts coded information from the input devices such as keyboards or from other computers over digital communication lines. The information is either stored in the computer's memory for later reference or

immediately used by ALU to perform the desired operation. The processing steps are determined by a program stored in the memory. Finally the results are sent back to outside world through output unit. All these operations are coordinated by the control unit.

The information handled by a computer may be either instruction or data. Instructions are the explicit commands that

- Govern the transfer of information within a computer as well as between the computer and its I/O devices.
- Specify the arithmetic & logic operations to be performed.

A list of instructions that performs a task is called a program. Usually the program is stored in the memory. The processor then fetches the instructions that make up the program from the memory one after another, and performs the desired operations. The computer is completely controlled by the stored program except for the external interruption.

Data are the numbers and encoded characters that are used as operands by the instructions.

Information handled by a computer must be encoded in a suitable format. Each number, character, or instruction is encoded as a string of binary digits called as bits, each having one of two possible values 0 or 1.

Alphanumeric characters are also expressed in the form binary codes.

Two commonly used codes are:

- **ASCII** (American Standard code for Information Interchange)
- **EBCDIC** (Extended Binary-Coded Decimal Interchange Code)

INPUT UNIT:

Computers accept coded information through input units, which read the data. The most well-known input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated in to corresponding binary code and transmitted over a cable to either the memory or the processor.

Many other kinds of input devices are available, including joysticks, trackballs, and mouse. These are often used as graphic input devices in conjunction with displays. Microphones can be used to capture audio input which is then sampled and converted into digital codes for storage and processing.

MEMORY UNIT:

The function of the memory unit is to store programs and data. There are two classes of storage, called primary and secondary.

Primary storage is a fast memory that operates at electronic speeds. Programs must be stored in the memory while they are being executed. The memory contains a large number of semiconductor storage cells, each capable of storing one bit of information. These cells are rarely read or written as individual cells but instead are processed in groups of fixed size called *words*. The memory is organized so that the contents of one word, containing n bits, can be stored or retrieved in one basic operation.

To provide easy access to any word in the memory, a distinct *address* is associated with each word location. Addresses are numbers that identify successive locations. A given word is accessed by specifying its address and issuing a control command that starts the storage or retrieval process.

The number of bits in each word is often referred to as the *word length* of the computer. Typical word lengths range from 16 to 64 bits.

Programs must reside in the memory during execution. Instructions and data can be written into the memory or read out under the control of the processor.

Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called *random-access memory* (RAM).

The time required to access one word is called the *memory access time*. This time is fixed, independent of the location of the word being accessed. It typically ranges from few nanoseconds (ns) to about 100 ns for modern RAM units. The small, fast RAM units are called caches. They are tightly coupled with the processor and are often contained on the same IC to achieve high performance.

Although primary storage is essential, it tends to be expensive. Thus additional, cheaper, secondary storage is used when large amounts of data and many programs have to be stored, particularly for information that is accessed infrequently.

ARITHMETIC & LOGIC UNIT

Most computer operations are executed in the *arithmetic and logic unit* (ALU) of the processor. Consider a typical example: Suppose two numbers located in the memory are to be added. They are brought into the processor, and the actual addition is carried out by the ALU. The sum may then be stored in the memory or retained in the processor for immediate use. Any other arithmetic or logic operation, for example, multiplication, division, or comparison of numbers, is initiated by bringing the required operands into the processor, where the operation is performed by the ALU. When operands are brought into the processor, they are stored in high-speed storage elements called *registers*. Each register can store one word of data. Access times to registers are somewhat faster than access times to the fastest cache unit in the memory hierarchy.

OUTPUT UNIT

The output unit is the counterpart of the input unit. Its function is to send processed results to the outside world. Examples are Monitor, printer etc.

CONTROL UNIT

The memory, arithmetic and logic, and input and output units store and process information and perform input and output operations. The operation of these units is coordinated by control unit. The control unit is effectively the nerve center that sends control signals to other units and senses their states.

Timing signals that govern the I/O transfers are generated by the control circuits. Timing signals are signals that determine when a given action is to take place. Data transfers between the processor and the memory are also controlled by the control unit through timing signals.

BASIC OPERATIONAL CONCEPTS

The activity in a computer is governed by instructions. To perform a given task, an appropriate program consisting of a list of instructions is stored in the memory. Individual instructions are brought from the memory into the processor, which executes the specified operations. Data to be used as operands are also stored in the memory. Consider the instruction

Add LOCA, R0

This instruction adds the operand at memory location LOCA to the operand in a register in the processor, R0, and places the sum into register R0. The original contents of location LOCA are preserved, whereas those of R0 are overwritten. This instruction requires the performance of several steps. First, the instruction is fetched from the memory into the processor. Next, the operand at LOCA is fetched and added to the contents of R0.

Finally, the resulting sum is stored in register R0.

Transfers between the memory and the processor are started by sending the address of the memory location to be accessed to the memory unit and issuing the appropriate control signals. The data are then transferred to or from the memory.

Figure 1.2 shows how the memory and the processor are connected.

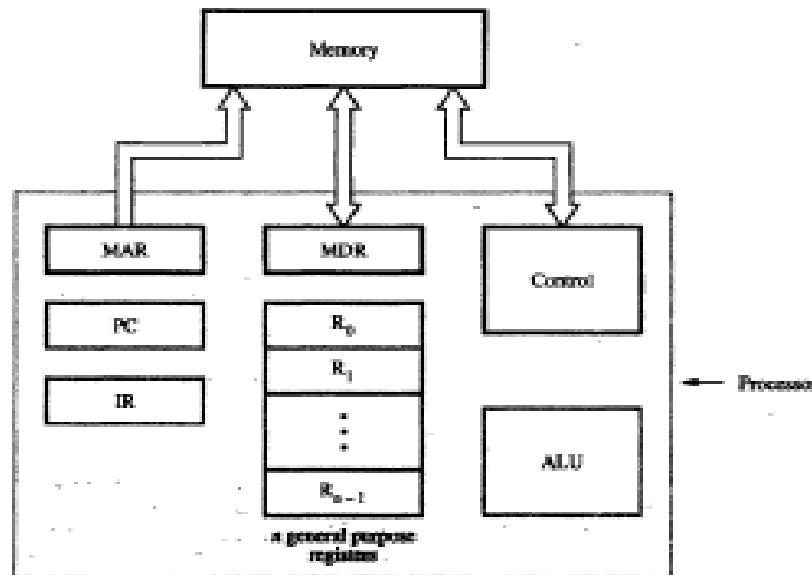


Figure 1.2 Connections between the processor and the memory.

In addition to the ALU and the control circuitry, the processor contains a number of registers used for several different purposes.

The *instruction register* (IR) holds the instruction that is currently being executed. Its output is available to the control circuits, which generate the timing signals that control the various processing elements involved in executing the instruction.

The *program counter* (PC) is another specialized register, which keeps track of the execution of a program. It contains the memory address of the next instruction to be fetched and executed. During the execution of an instruction, the contents of the PC are updated to correspond to the

address of the next instruction to be executed. The PC always *points* to the next instruction that is to be fetched from the memory.

In addition to IR and PC, Processor also contains *n general-purpose registers*, R0 through Rn-1.

Finally, two registers facilitate communication with the memory. These are the *memory address register* (MAR) and the *memory data register* (MDR). The MAR holds the address of the location to be accessed. The MDR contains the data to be written into or read out of the addressed location.

The following are steps involved in executing a program.

- Execution of the program starts when the PC is set to point to the first instruction of the program.
- The contents of the PC are transferred to the MAR and a Read control signal is sent to the memory.
- After the time required to access the memory elapses, the addressed word is read out of the memory and loaded into the MDR.
- The contents of the MDR are transferred to the IR. Now the instruction is ready for decoding and execution.
- If the instruction involves an operation to be performed by the ALU, it is necessary to obtain the required operands. If an operand resides in the memory, it has to be fetched by sending its address to the MAR and initiating a Read cycle.
- When the operand has been read from the memory into the MDR, it is transferred from the MDR to the ALU. After one or more operands are fetched in this way, the ALU can perform the desired operation.
- If the result of this operation is to be stored in the memory, then the result is sent to the MDR. The address of the location where the result is to be stored is sent to the MAR, and a Write cycle is initiated.
- At some point during the execution of the current instruction, the

contents of the PC are incremented so that the PC points to the next instruction to be executed. Thus, as soon as the execution of the current instruction is completed, a new instruction fetch may be started.

BUS STRUCTURES

To form an operational system, the functional units of the computer must be connected in some organized way.

To achieve a reasonable speed of operation, a computer must be organized so that all its units can handle one full word of data at a given time. When a word of data is transferred between units, all its bits are transferred in parallel, that is, the bits are transferred simultaneously over many wires, or lines, one bit per line. A group of lines that serves as a connecting path for several devices is called a *bus*.

In addition to the lines that carry the data, the bus must have lines for address and control information.

Single Bus Structure

The simplest way to interconnect functional units is to use a *Single bus*, as shown below.

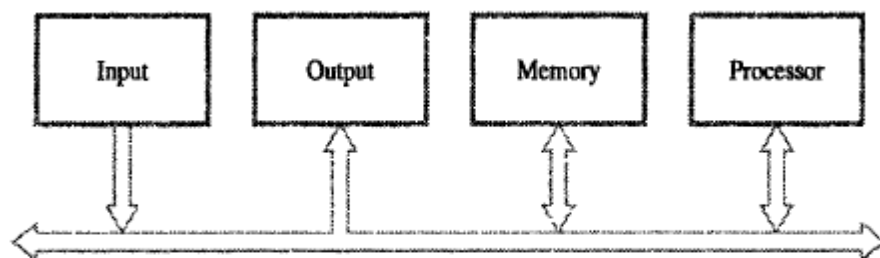


Figure 1.3 Single-bus structure.

All units are connected to the same bus. Since the bus can be used for only

one transfer at a time, only two units can actively use the bus at any given time.

Advantages:

- Simple
- Low cost
- Flexibility to attach peripheral devices.

Disadvantages:

- Concurrent operations cannot be performed.
- Lower performance

Multiple Bus Structure

- Multiple Buses are used to interconnect functional units.

Advantage:

- Concurrent operations can be performed
- Higher performance

Disadvantages:

- Complex
- Costlier

The devices connected to a bus vary widely in their speed of operation. Some electromechanical devices, such as keyboards and printers, are relatively slow. Others, like magnetic or optical disks, are considerably faster. Memory and processor units operate at electronic speeds, making them the fastest parts of a computer. Because all these devices must communicate with each other over a

bus, an efficient transfer mechanism is required to smooth out the differences in timing among processors, memories, and external devices.

A common approach is to include *buffer registers* with the devices to hold the information during transfers. Consider the task of printing a file, The processor sends the contents of the file over the bus to the printer buffer. Since the buffer is an electronic register, this transfer requires relatively little time. Once the buffer is loaded, the printer can start printing without further intervention by the processor. The bus and the processor are no longer needed and can be used for other activities. The printer continues print and is not available for further transfers until this process is completed. Thus, buffer registers smooth out timing differences among processors, memories, and I/O devices. They prevent a high-speed processor from being locked to a slow I/O device during a sequence of data transfers.

PERFORMANCE

The most important measure of the performance of a computer is how quickly it can execute programs. The speed with which a computer executes programs is affected by:

1. the design of its hardware
2. its machine language instructions.
3. performance is also affected by the compiler that translates programs into machine language.

For best performance, it is necessary to design the compiler, the machine instruction set, and the hardware in a coordinated way.

PROCESSOR CLOCK

Processor circuits are controlled by a timing signal called a *clock*. The clock defines regular time intervals, called *clock cycles*. To execute a machine instruction, the processor divides the action to be performed into a sequence of basic steps, such that each step can be completed in one clock cycle.

The length P of one clock cycle is an important parameter that affects processor performance. Its inverse is the clock rate,

$$R = 1 / P,$$

which is measured in cycles per second.

BASIC PERFORMANCE EQUATION

Let T be the processor time required to execute a program that has been prepared in some high-level language. The compiler generates a machine language object program that corresponds to the source program. Assume that complete execution of the program requires the execution of N machine language instructions. The number N is the actual number of instruction executions, and is not necessarily equal to the number of machine instructions in the object program. Some instructions may be executed more than once, which is the case for instructions inside a program loop. Others may not be executed at all, depending on the input data used. Suppose that the average number of basic steps needed to execute one machine instruction is S , where each basic step is completed in one clock cycle. If the clock rate is R cycles per second, the program execution time is given by

$$T = (N * S) / R$$

This is often referred to as the *basic performance equation*.

To achieve high performance, the computer designer must seek ways to reduce

the value of T , which means reducing N and S , and increasing R . The value of N is reduced if the source program is compiled into fewer machine instructions. The value of S is reduced if instructions have a smaller number of basic steps to perform or if the execution of instructions is overlapped. Using a higher-frequency clock increases the value of R , which means that the time required to complete a basic execution step is reduced.

There are two possibilities for increasing the clock rate, R .

- First, improving the *integrated-circuit* (IC) technology makes logic circuits faster, which reduces the time needed to complete a basic step. This allows the clock period, P , to be reduced and the clock rate, R , to be increased.
- Second, reducing the amount of processing done in one basic step also makes it possible to reduce the clock period, P . However, if the actions that have to be performed by an instruction remain the same; the number of basic steps needed may increase.

PERFORMANCE MEASUREMENT

The performance of a computer is mainly affected by the execution time, T . But computing the value of T is not simple. Moreover, parameters such as the clock speed and various architectural features are not reliable indicators of the expected performance.

For these reasons, the computer community adopted the idea of measuring computer performance using benchmark programs. The performance measure is the time it takes a computer to execute a given benchmark.

A nonprofit organization called System Performance Evaluation Corporation (SPEC) selects the benchmark programs. The programs selected range from game playing, compiler, and database applications to numerically intensive

programs in astrophysics and quantum chemistry. In each case, the program is compiled for the computer under test, and the running time on a real computer is measured. The same program is also compiled and run on one computer selected as a reference. The SPEC rating is computed as follows

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

Thus a SPEC rating of 50 means that the computer under test is 50 times faster than the reference computer. The test is repeated for all the programs in the SPEC suite, and the geometric mean of the results is computed.

Let SPEC_i be the rating for program i in the suite. The overall SPEC rating for the computer is given by

$$\text{SPEC rating} = \left(\prod_{i=1}^n \text{SPEC}_i \right)^{\frac{1}{n}}$$

Where n is the number of programs in the suite.

Because the actual execution time is measured, the SPEC rating is a measure of the combined effect of all the factors affecting performance, including the compiler, the operating system, the processor, and the memory of the computer being tested.

Problems:

1. For a RISC machine, the effective value of S is 1.25 and the average value of N is 200. If the clock rate is 500 MHz, calculate the total program execution time.

Soln: $T = (N \times S)/R = (200 \times 1.25)/(500 \times 10^6) = 0.5 \mu\text{s}$

2. The effective value of S for a RISC machine is 1.2 and for CISC is 1.5. Both machines have the same clock rate R. The time for execution on CISC machine is to be no longer than that of RISC machine. For this to happen, what is the largest allowable value for N, i.e., no. of instructions executed on CISC machine expressed as a percentage of N value for the RISC machine.

Soln: Given, $S_C=1.5$, $S_R=1.2$, $R_C=R_R=R$

$$(1.2 \times N_R)/R = (1.5 \times N_C)/R$$

$$1.2N_R = 1.5N_C$$

$$N_C/N_R = 0.8$$

- *No. of instructions executed on CISC machine is 80% of the no. of instructions executed on RISC machine.*

3. The effective value of S for a RISC machine is 1.2 and for CISC is 1.5. The clock rate R for RISC machine is 20% higher than that of CISC machine. The time for execution on CISC machine is to be no longer than that of RISC machine. For this to happen, what is the largest allowable value for N, i.e., no. of instructions executed on CISC machine expressed as a percentage of N value for the RISC machine.

Soln: Given, $S_C=1.5$, $S_R=1.2$

$$(1.2 \times N_R)/R_R = (1.5 \times N_C)/R_C$$

$$1.2N_R/1.2 = 1.5N_C/1$$

$$N_C/N_R = 0.666$$

- *No. of instructions executed on CISC machine is 66.6% of the no. of instructions executed on RISC machine.*

4. Assuming that the reference computer is Ultra SPARC10 workstation with 300 MHz Ultra SPARC processor. A company has to purchase 1000 new computers, hence ordered testing of new computers with SPEC 2000. Following observations were made:

<u>Programs</u>	<u>Runtime on reference computer</u>	<u>Runtime on new computer</u>
1	50 mins	5 mins
2	75 mins	4 mins
3	60 mins	6 mins
4	30 mins	3 mins

The company system manager will place the order for purchasing new computers only if the overall SPEC rating is at least 12. After the said test, will system manager place order for purchase of new computer?

Soln: $SPEC_1 = 50/5 = 10$

$$SPEC_2 = 75/4 = 18.75$$

$$SPEC_3 = 60/6 = 10$$

$$SPEC_4 = 30/3 = 10$$

$$\text{Overall SPEC rating} = (\prod_1^n SPEC_i)^{1/n} = (48.75)^{1/4} = 11.70.$$

Since overall SPEC rating is 11.70, the purchase order will not be placed.

5. List the steps needed to execute the machine instruction Add LOCA,R0 in terms of transfers between the components of the processor and some control commands. Assume that the instruction itself is stored in the memory at location INSTR and that this address is initially in register PC. Also include the steps needed to update the contents of PC from INSTR to INSTR+1 so that the next instruction can be fetched.

Soln:

- Transfer the contents of PC to MAR.
- Issue Read signal to memory and wait until requested word is transferred into MDR.
- Transfer MDR content to IR and decode it.
- Transfer the address LOCA to MAR.

- Issue Read signal and wait until MDR is loaded with the data.
- Transfer MDR content to the ALU.
- Transfer contents of R0 to the ALU.
- Perform addition of the two operands in the ALU and transfer the result into R0.
- Transfer the contents of PC to ALU.
- Add 4 to operand in ALU and transfer incremented address to PC.

NUMBER REPRESENTATION

Consider an n -bit vector

$$B = b_{n-1} \dots b_1 b_0$$

Where, $b_i = 0$ or 1 for $0 \leq i \leq n-1$. This vector can represent unsigned integer values V in the range 0 to $2^n - 1$, where,

$$V(B) = b_{n-1} * 2^{n-1} + \dots + b_1 * 2^1 + b_0 * 2^0$$

There is a need to represent both positive and negative numbers. **Three systems are used for representing such numbers:**

- Sign-and-magnitude
- 1's-complement
- 2's-complement
- In all three systems, the leftmost bit is 0 for positive numbers and 1 for negative numbers. The following figure illustrates all three representations using 4-bit numbers.

B $b_3 b_2 b_1 b_0$	Values represented		
	Sign and magnitude	1's complement	2's complement
0 1 1 1	+7	+7	+7
0 1 1 0	+6	+6	+6
0 1 0 1	+5	+5	+5
0 1 0 0	+4	+4	+4
0 0 1 1	+3	+3	+3
0 0 1 0	+2	+2	+2
0 0 0 1	+1	+1	+1
0 0 0 0	+0	+0	+0
1 0 0 0	-0	-7	-8
1 0 0 1	-1	-6	-7
1 0 1 0	-2	-5	-6
1 0 1 1	-3	-4	-5
1 1 0 0	-4	-3	-4
1 1 0 1	-5	-2	-3
1 1 1 0	-6	-1	-2
1 1 1 1	-7	-0	-1

Figure 2.1 Binary, signed-integer representations.

- Positive values have identical representations in all systems, but negative values have different representations.
- In the **sign-and-magnitude** system, negative values are represented by changing the most significant bit from 0 to 1. For example, +5 is represented by 0101, and -5 is represented by 1101.
- In **1's-complement** representation, negative values are obtained by complementing each bit of the corresponding positive number. Thus, the representation for -3 is obtained by complementing each bit in the vector 0011 to yield 1100. The operation of forming the 1's-complement of a given number is equivalent to subtracting that number from $2^n - 1$, that is, from 1111 in case of 4-bit numbers.
- Finally, in the **2's-complement** system, forming the 2's-complement of a number is done by subtracting that number from 2^n . Hence, the 2's-complement of a number is obtained by adding 1 to the 1's-complement of that number.
- There are distinct representations for +0 and - 0 in both the sign-and-magnitude and 1's-complement systems, but the 2's-complement system has only one representation for 0. For 4-bit numbers, the value -8 is representable in the 2's-complement system but not in the other systems.

ADDITION OF POSITIVE NUMBERS

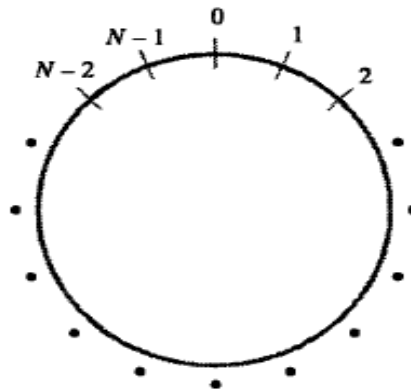
Consider adding two 1-bit numbers. The results are shown in following figure. The sum of 1 and 1 requires the 2-bit vector 10 to represent the value 2. We say that the *sum* is 0 and the *carry-out* is 1.

$$\begin{array}{cccc}
 \begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array} &
 \begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array} &
 \begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array} &
 \begin{array}{r} 1 \\ + 1 \\ \hline 10 \\ \uparrow \\ \text{Carry-out} \end{array}
 \end{array}$$

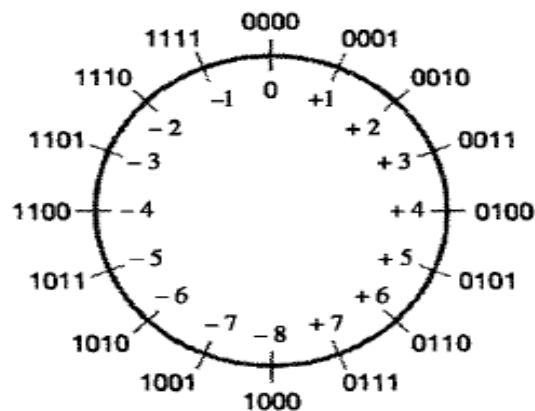
Figure 2.2 Addition of 1-bit numbers.

ADDITION AND SUBTRACTION OF SIGNED NUMBERS

- There are three systems for representing positive and negative numbers. These systems differ only in the way they represent negative values.
- The sign-and-magnitude system is the simplest representation, but it is also the most awkward for addition and subtraction operations. The 1's-complement method is somewhat better. The **2's-complement system** is the most efficient method for performing addition and subtraction operations.
- Consider addition modulo N (written as mod N). The description of addition mod N of positive integers is represented by a circle with the N values, 0 through $N - 1$, marked along its perimeter.



(a) Circle representation of integers mod N



(b) Mod 16 system for 2's-complement numbers

Figure 2.3 Modular number systems and the 2's-complement system.

- Consider the case $N = 16$. The operation $(7+4) \bmod 16$ yields the value 11. To perform this operation graphically, locate 7 on the circle and then move 4 units in the clockwise direction to arrive at the answer 11. Similarly, $(9 + 14) \bmod 16 = 7$; this is modeled on the circle by locating 9 and moving 14 units in the clockwise direction to arrive at the answer 7. This technique works for the computation of $(a + b) \bmod 16$ for any positive numbers a and b , that is, to perform addition, locate a and move b units in the clockwise direction to arrive at $(a + b) \bmod 16$.
- Consider the addition of +7 to -3. The 2's-complement representation for these numbers is 0111 and 1101, respectively. To add these numbers, locate 0111 on the circle then move 1101 (13) steps in the clockwise direction to arrive at 0100, which yields the correct answer of +4.

$$\begin{array}{r}
 0111 \\
 + 1101 \\
 \hline
 10100 \\
 \uparrow \\
 \text{Carry-out}
 \end{array}$$

If we ignore the carry-out from the fourth bit position, we obtain the correct answer.

- The rules for addition and subtraction of n -bit signed numbers using the 2's-complement representation system are:**
 - To *add* two numbers, add their n -bit representations, ignoring the carry-out signal from the *most significant bit* (MSB) position. The sum will be the algebraically correct value in the 2's-complement representation as long as the answer is in the range -2^{n-1} through $+2^{n-1} - 1$.
 - To subtract two numbers X and Y , that is, to perform $X - Y$, form the 2'scomplement of Y and then add it to X , as in rule 1. Again, the result will be the algebraically correct value in the 2's-complement representation system if the answer is in the range -2^{n-1} through $+2^{n-1} - 1$.

- The simplicity of either adding or subtracting signed numbers in 2's-complement representation is the reason why this number representation is used in modern computers.

• **Examples :**

(a)	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{r} (+2) \\ (+3) \\ \hline (+5) \end{array}$	(b)	$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$	$\begin{array}{r} (+4) \\ (-6) \\ \hline (-2) \end{array}$
(c)	$\begin{array}{r} 1011 \\ + 1110 \\ \hline 1001 \end{array}$	$\begin{array}{r} (-5) \\ (-2) \\ \hline (-7) \end{array}$	(d)	$\begin{array}{r} 0111 \\ + 1101 \\ \hline 0100 \end{array}$	$\begin{array}{r} (+7) \\ (-3) \\ \hline (+4) \end{array}$
(e)	$\begin{array}{r} 1101 \\ - 1001 \\ \hline \end{array}$	$\begin{array}{r} (-3) \\ (-7) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1101 \\ + 0111 \\ \hline 0100 \end{array}$	$\begin{array}{r} \\ \\ \hline (+4) \end{array}$
(f)	$\begin{array}{r} 0010 \\ - 0100 \\ \hline \end{array}$	$\begin{array}{r} (+2) \\ (+4) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0010 \\ + 1100 \\ \hline 1110 \end{array}$	$\begin{array}{r} \\ \\ \hline (-2) \end{array}$
(g)	$\begin{array}{r} 0110 \\ - 0011 \\ \hline \end{array}$	$\begin{array}{r} (+6) \\ (+3) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0110 \\ + 1101 \\ \hline 0011 \end{array}$	$\begin{array}{r} \\ \\ \hline (+3) \end{array}$
(h)	$\begin{array}{r} 1001 \\ - 1011 \\ \hline \end{array}$	$\begin{array}{r} (-7) \\ (-5) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$	$\begin{array}{r} \\ \\ \hline (-2) \end{array}$
(i)	$\begin{array}{r} 1001 \\ - 0001 \\ \hline \end{array}$	$\begin{array}{r} (-7) \\ (+1) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1001 \\ + 1111 \\ \hline 1000 \end{array}$	$\begin{array}{r} \\ \\ \hline (-8) \end{array}$
(j)	$\begin{array}{r} 0010 \\ - 1101 \\ \hline \end{array}$	$\begin{array}{r} (+2) \\ (-3) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{r} \\ \\ \hline (+5) \end{array}$

Figure 2.4 2's-complement add and subtract operations.

OVERFLOW IN INTEGER ARITHMETIC

- In the 2's-complement number representation system, n bits can represent values in the range -2^{n-1} to $+2^{n-1} - 1$. For 4 bit number system, the range of numbers that can be represented is -8 through +7. If the result of an arithmetic operation is outside the representable range, then we say that *arithmetic overflow* has occurred.
- When adding unsigned numbers, the carry-out, cn , from the most significant bit position serves as the overflow indicator. But this does not work for adding signed numbers. For example, when using 4-bit signed numbers, if we try to add the numbers +7 and +4, the output sum vector, S , is 1011, which is the code for -5, an incorrect result. The carry-out signal from the MSB position is 0. Similarly, if we try to add -4 and -6, we get $S = 0110 = +6$, another incorrect result, and in this case, the carry-out signal is 1. Thus, overflow may occur if both summands have the same sign. Clearly, the addition of numbers with different signs cannot cause overflow. This leads to the following conclusions:
 1. Overflow can occur only when adding two numbers that have the same sign.
 2. The carry-out signal from the sign-bit position is not a sufficient indicator of overflow when adding signed numbers.
- A simple way to detect overflow is to examine the signs of the two summands X and Y and the sign of the result. When both operands X and Y have the same sign, an overflow occurs when the sign of S is not the same as the signs of X and Y .

MEMORY LOCATIONS AND ADDRESSES

Information is stored in the memory. The memory consists of many millions of storage *cells*, each of which can store a bit of information having the value 0 or 1. Because a single bit represents a very small amount of information, bits are not handled individually. The usual approach is to deal with them in groups of fixed size. For this purpose, the memory is organized so that a group of n bits can be stored or retrieved in a single, basic operation. Each group of n bits is referred to as a *word* of information, and n is called the *word length*. The memory of a computer can be schematically represented as a collection of words as shown in Figure.

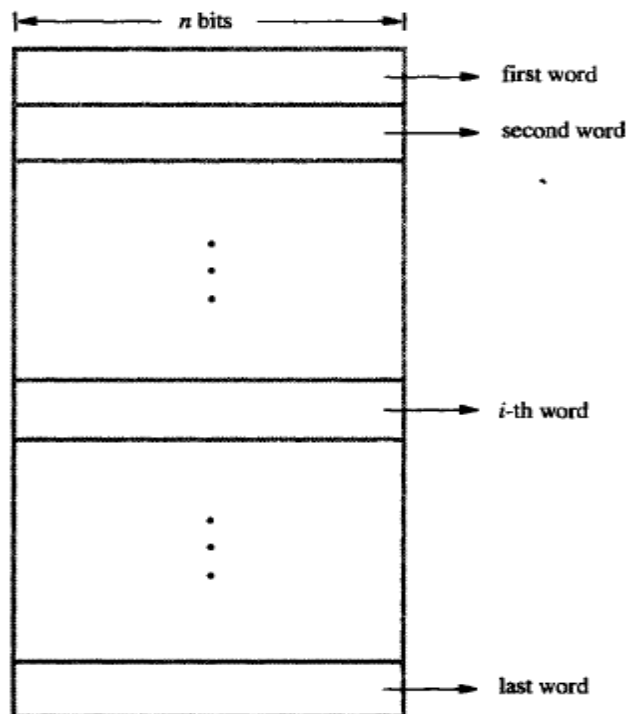


Figure 2.5 Memory words.

The word length of the computer typically ranges from 16 to 64 bits. If the word length of a computer is 32 bits, a single word can store a 32-bit 2's-complement number or four ASCII characters, each occupying 8 bits. A unit of 8 bits is called a *byte*. Machine instructions may require one or more words for their representation.

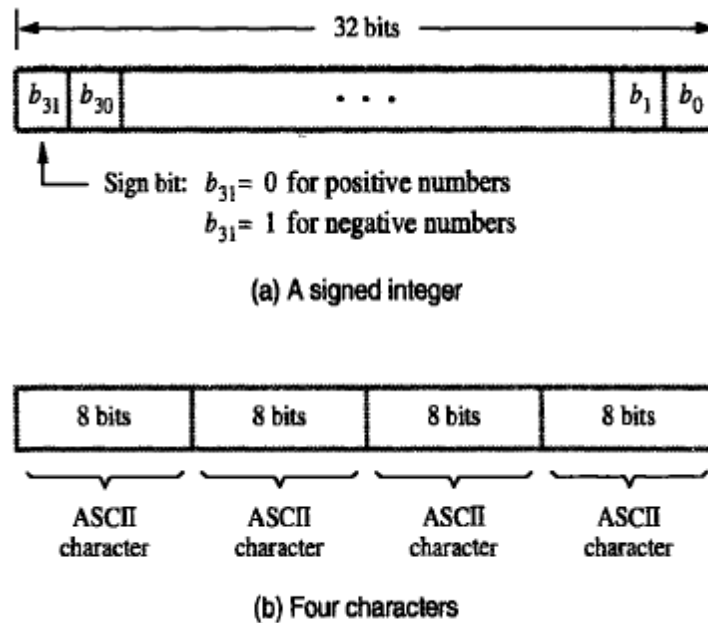


Figure 2.6 Examples of encoded information in a 32-bit word.

- Accessing the memory to store or retrieve a single item of information, either a word or a byte, requires distinct names or *addresses* for each item location. It is customary to use numbers from 0 through 2^k-1 , for some suitable value of k . The 2^k addresses constitute the *address space* of the computer, and the memory can have up to 2^k addressable locations. For example, a 24-bit address generates an address space of 2^{24} (16,777,216) locations. This number is usually written as 16M (16 Mega), where 1M is the number 2^{20} (1,048,576). A 32-bit address creates an address space of 2^{32} or 4G (4 Giga) locations, where 1G is 2^{30} .

BYTE ADDRESSABILITY

- The memory consists of three basic information quantities: the **bit**, **byte**, and **word**. A byte is always 8 bits, but the word length typically ranges from 16 to 64 bits. It is impractical to assign distinct addresses to individual bit locations in the memory. The most practical assignment is to have successive addresses refer to successive byte locations in the memory. The term *byte-addressable memory* is used for this assignment.

Byte locations have addresses 0, 1, 2, Thus, if the word length of the machine is 32 bits, successive words are located at addresses 0, 4, 8, . . . with each word consisting of four bytes.

BIG-ENDIAN AND LITTLE-ENDIAN ASSIGNMENTS

- There are two ways that byte addresses can be assigned across words, as shown in Figure.
- The name **big-endian** is used when lower byte addresses are used for the more significant bytes (the leftmost bytes) of the word.
- The name **little-endian** is used for the opposite ordering, where the lower byte addresses are used for the less significant bytes (the rightmost bytes) of the word.
- Both little-endian and big-endian assignments are used in commercial machines. In both cases, byte addresses 0, 4, 8, . . . are taken as the addresses of successive words in the memory and are the addresses used when specifying memory read and write operations for words.

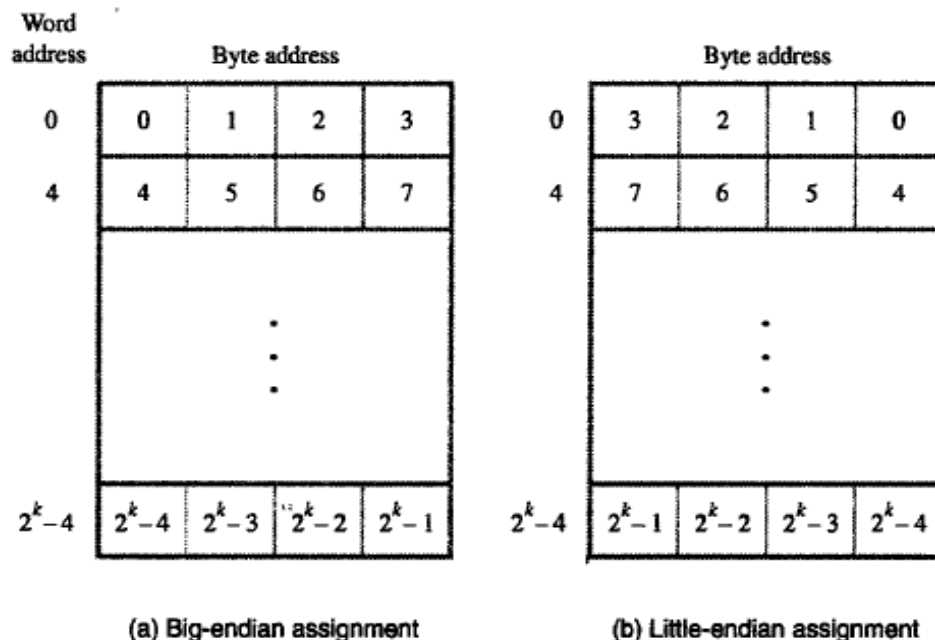


Figure 2.7 Byte and word addressing.

WORD ALIGNMENT

- If the word length is 32-bit, then word boundaries occur at addresses 0, 4, 8, We say that the word locations have *aligned* addresses. In general, **words are said to be aligned** in memory if they begin at a byte address that is a multiple of the number of bytes in a word. In general, the number of bytes in a word is a power of 2. Hence, if the word length is 16 (2 bytes), aligned words begin at byte addresses 0, 2, 4, . . . , and for a word length of 64 (23 bytes), aligned words begin at byte addresses 0, 8, 16, If words begin at an arbitrary byte address then they are said to have **unaligned addresses**.

MEMORY OPERATIONS

- Both program instructions and data operands are stored in the memory. To execute an instruction, the processor control circuits must cause the word (or words) containing the instruction to be transferred from the memory to the processor. Operands and results must also be moved between the memory and the processor. Thus, two basic operations involving the memory are needed, namely, **Load** (or *Read* or *Fetch*) and **Store** (or *Write*).
- The **Load operation** transfers a copy of the contents of a specific memory location to the processor. The memory contents remain unchanged. To start a Load operation, the processor sends the address of the desired location to the memory and requests that its contents be read. The memory reads the data stored at that address and sends them to the processor.
- The **Store operation** transfers an item of information from the processor to a specific memory location, destroying the initial contents of that location. The processor sends the address of the desired location to the memory, together with the data to be written into that location.

INSTRUCTIONS AND INSTRUCTION SEQUENCING

- The tasks carried out by a computer program consist of a sequence of small steps, such as adding two numbers, testing for a particular condition, reading a character from the keyboard, or sending a character to be displayed on a display screen.
- A computer must have instructions capable of performing four types of operations:
 - Data transfers between the memory and the processor registers
 - Arithmetic and logic operations on data
 - Program sequencing and control
 - I/O transfers

REGISTER TRANSFER NOTATION

- To transfer the information from one location in the computer to another the following locations are involved: memory locations, processor registers, or registers in the I/O subsystem.
- The location is identified by a symbolic name. For example, names for the addresses of memory locations may be LOC, PLACE, A, VAR2; processor register names may be R0, R5; and I/O register names may be DATAIN, OUTSTATUS, and so on.
- The contents of a location are denoted by placing square brackets around the name of the location. Thus, the expression

$R1 \leftarrow [LOC]$

means that the contents of memory location LOC are transferred into processor register R1.

- Consider the operation that adds the contents of registers R1 and R2, and then places their sum into register R3. This action is indicated as

$R3 \leftarrow [R1] + [R2]$

- This type of notation is known as **Register Transfer Notation (RTN)**. The right-hand side of an RTN expression always denotes a value, and

the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location.

ASSEMBLY LANGUAGE NOTATION

- There is another type of notation to represent machine instructions and programs. For this, we use an *assembly language* format. For example, an instruction that causes the transfer from memory location LOC to processor register R1, is specified by the statement

Move LOC,R1

- The contents of LOC are unchanged by the execution of this instruction, but the old contents of register R1 are overwritten.
- The addition of two numbers contained in processor registers R1 and R2 and placing their sum in R3 can be specified by the assembly language statement

Add R1,R2,R3

BASIC INSTRUCTION TYPES

The operation of adding two numbers is a fundamental capability in any computer. The statement

$$C = A + B$$

in a high-level language program is a command to the computer to add the current values of the two variables called A and B, and to assign the sum to a third variable, C. When the program containing this statement is compiled, the three variables, A, B, and C, are assigned to distinct locations in the memory. The contents of these locations represent the values of the three variables. Hence, the above high-level language statement requires the action

$$C \leftarrow [A] + [B]$$

to take place in the computer. To carry out this action, the contents of memory locations A and B are fetched from the memory and transferred into the

processor where their sum is computed. This result is then sent back to the memory and stored in location C.

If this action is to be performed by a single machine instruction and the instruction contains the memory addresses of the three operands—A, B, and C then *three-address* instruction can be represented symbolically as

Add A,B,C

Operands A and B are called the *source* operands, C is called the *destination* operand, and Add is the operation to be performed on the operands.

Three-address instruction:

- A general instruction of this type has the format

Operation Source1,Source2,Destination

- If k bits are needed to specify the memory address of each operand, the encoded form of the above instruction must contain $3k$ bits for addressing purposes in addition to the bits needed to denote the Add operation.
- For a modern processor with a 32-bit address space, a 3-address instruction is too large to fit in one word for a reasonable word length.

Two-address instruction:

- A general instruction of this type has the format

Operation Source, Destination

- An Add instruction of this type is

Add A,B

which performs the operation $B \leftarrow [A] + [B]$. When the sum is calculated, the result is sent to the memory and stored in location B, replacing the original contents of this location. This means that operand B is both a source and a destination.

- The problem in two address instruction is one of the operand is destroyed. The problem can be solved by using another two address instruction that copies the contents of one memory location into another.

Move B,C

which performs the operation $C \leftarrow [B]$, leaving the contents of location B unchanged.

The operation $C \leftarrow [A] + [B]$ can now be performed by the two-instruction sequence

Move B,C

Add A,C

- Two-address instructions will not normally fit into one word for usual word lengths and address sizes.

One-address instruction:

- Another possibility is to have machine instructions that specify only one memory operand. The second operand is implicitly specified in the instruction. A processor register, called the *accumulator*, is used for this purpose.
- Examples:

Add A

This instruction adds the contents of memory location A to the contents of the accumulator register and place the sum back into the accumulator.

Load A

The Load instruction copies the contents of memory location A into the accumulator.

Store A

The Store instruction copies the contents of the accumulator into memory location A.

- Now the operation $C \leftarrow [A] + [B]$ can be performed by

Load A

Add B

Store C

The operand specified in the instruction may be a source or a destination, depending on the instruction. In the Load instruction, address A specifies the source operand, and the destination location, the accumulator, is implied. On the other hand, C denotes the destination location in the Store instruction, whereas the source, the accumulator, is implied.

Zero-address instruction:

- It is also possible to use instructions in which the locations of all operands are defined implicitly. Such instructions are found in machines that store operands in a structure called a pushdown stack.
- Example: Consider the C statement, $C = A + B$

PUSH A

PUSH B

ADD

POP C

INSTRUCTION EXECUTION AND STRAIGHT-LINE SEQUENCING

Consider the task, $C \leftarrow [A] + [B]$. Figure shows a possible program segment for this task as it appears in the memory of a computer. Assume that the computer allows one memory operand per instruction, has a number of processor registers, word length is 32 bits and the memory is byte addressable. The three instructions of the program are in successive word locations, starting at location i . Since each instruction is 4 bytes long, the second and third instructions start at addresses $i + 4$ and $i + 8$.

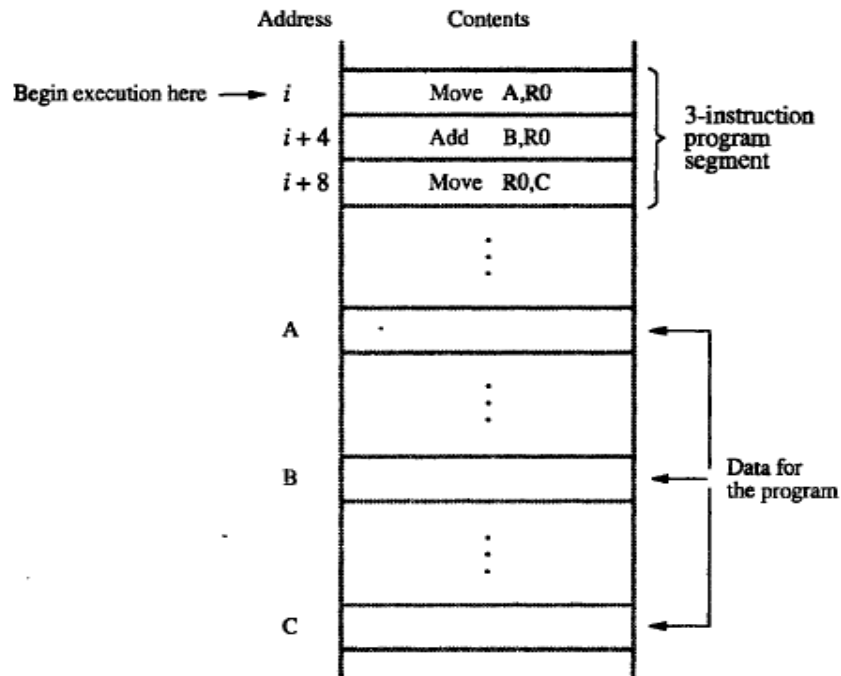


Figure 2.8 A program for $C \leftarrow [A] + [B]$.

- The processor contains a register called the *program counter (PC)*, which holds the address of the instruction to be executed next. To begin executing a program, the address of its first instruction (i) must be placed into the PC. Then, the *processor control circuits* use the information in the PC to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called *straight-line sequencing*. During the execution of each instruction, the PC is incremented by 4 to point to the next instruction.
- Executing a given instruction is a *two-phase procedure*.
 - In the first phase, called *instruction fetch*, the instruction is fetched from the memory location whose address is in the PC. This instruction is placed in the *instruction register (IR)* in the processor.
 - At the start of the second phase, called *instruction execute*, the instruction in IR is examined to determine which operation is to be performed. The specified operation is then performed by the processor. This often involves fetching operands from the memory or from processor registers, performing an arithmetic or logic

operation, and storing the result in the destination location. At some point during this two-phase procedure, the contents of the PC are advanced to point to the next instruction. When the execute phase of an instruction is completed, the PC contains the address of the next instruction, and a new instruction fetch phase can begin.

BRANCHING

- Consider the task of adding a list of n numbers. The program is shown in figure. The addresses of the memory locations containing the n numbers are symbolically given as NUM1, NUM2, . . . , NUM n , and a separate Add instruction is used to add each number to the contents of register R0. After all the numbers have been added, the result is placed in memory location SUM.

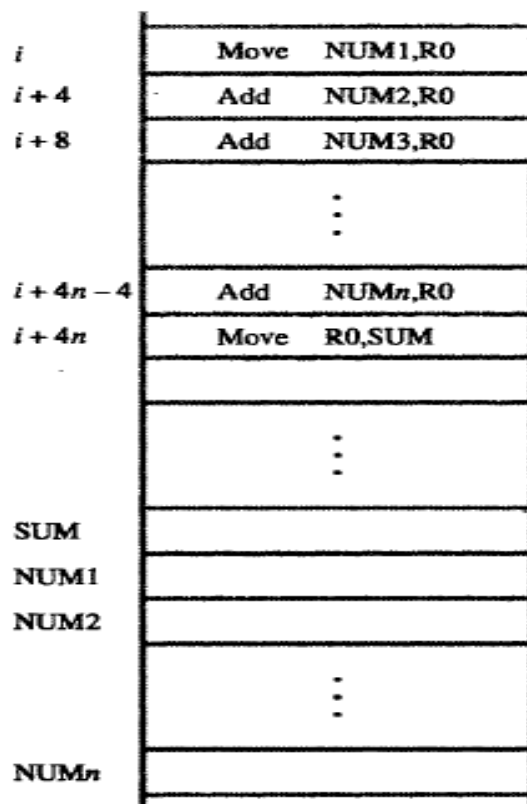


Figure 2.9 A straight-line program for adding n numbers.

- Instead of using a long list of Add instructions, it is possible to place a single Add instruction in a program loop, as shown below. The loop is a straight-line sequence of instructions executed as many times as needed. It starts at location LOOP and ends at the instruction Branch>0. During each pass through this loop, the address of the next list entry is determined, and that entry is fetched and added to R0.

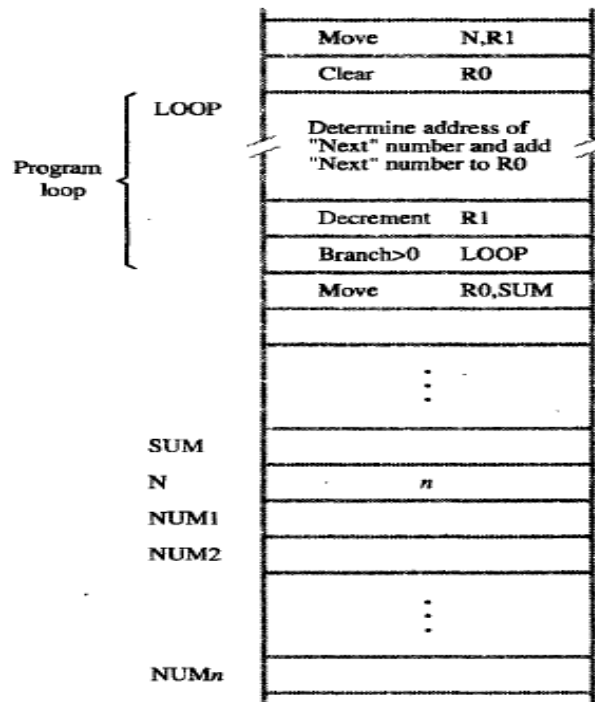


Figure 2.10 Using a loop to add n numbers.

- Assume that the number of entries in the list, n , is stored in memory location N, as shown. Register R1 is used as a counter to determine the number of times the loop is executed. Hence, the contents of location N are loaded into register R1 at the beginning of the program. Then, within the body of the loop, the instruction

Decrement R1

reduces the contents of R1 by 1 each time through the loop. Execution of the loop is repeated as long as the result of the decrement operation is greater than zero.

- Consider the *branch* instruction. This type of instruction loads a new value into the program counter. As a result, the processor fetches and executes the instruction at this new address, called the *branch target*, instead of the instruction at the location that follows the branch instruction in sequential address order.
- A *conditional branch* instruction causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

In the program, the instruction

Branch>0 LOOP

(branch if greater than 0) is a conditional branch instruction that causes a branch to location LOOP if the result of the immediately preceding instruction, which is the decremented value in register R1, is greater than zero. This means that the loop is repeated as long as there are entries in the list that are yet to be added to R0. At the end of the n th pass through the loop, the Decrement instruction produces a value of zero, and, hence, branching does not occur. Instead, the Move instruction is fetched and executed. It moves the final result from R0 into memory location SUM.

CONDITION CODES

- The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions. This is accomplished by recording the required information in individual bits, often called *condition code flags*. These flags are usually grouped together in a special processor register called the *condition code register* or *status register*. Individual condition code flags are set to 1 or cleared to 0, depending on the outcome of the operation performed.

- **Four commonly used flags are**

- **N (negative)** Set to 1 if the result of an arithmetic operation is negative; otherwise, cleared to 0.
- **Z (zero)** Set to 1 if the result of an arithmetic or logic operation is 0; otherwise, cleared to 0.
- **V (overflow)** Set to 1 if arithmetic overflow occurs i.e., if the result of an arithmetic operation is outside the representable range (-2^{n-1} to $+2^{n-1} - 1$ where n is the no. of bits used to represent the operands); otherwise, cleared to 0
- **C (carry)** Set to 1 if a carry occurs from the most significant bit position during an arithmetic operation.

***** End of Unit - 1 *****